

Project-1: Multi-dimensional Data Indexing and Similarity Query Processing

Πολυδιάστατες Δομές Δεδομένων

Πάτρα, 2026

ΜΑΛΛΙΑΡΟΥ - ΡΑΪΖΗ ZHNOBIA AM 1080451
ΠΕΤΡΟΠΟΥΛΟΥ ΚΩΝΣΤΑΝΤΙΝΑ-ΜΑΡΙΑ AM 1093479

Περιεχόμενα

<i>Εισαγωγή.....</i>	<i>2</i>
<i>Περιγραφή Δομών Δεδομένων.....</i>	<i>3</i>
KD-Tree	3
Quad Tree.....	3
Range Tree	3
R-Tree	4
Locality-Sensitive Hashing (LSH).....	4
Συνδυασμός Δομών με LSH	4
Υλοποίηση ανά Σχήμα Αναζήτησης.....	5
<i>Τεχνική Υλοποίηση</i>	<i>5</i>
Προεπεξεργασία Dataset.....	5
KD-Tree – Αναδρομική Κατασκευή	6
Quad Tree – Γεωμετρικός Διαχωρισμός.....	7
Range Tree – Ταξινόμηση και Αναζήτηση.....	8
R-Tree – 5D Ευρετηρίαση με Minimum Bounding Rectangles (MBRs).....	8
Locality-Sensitive Hashing (LSH).....	9
<i>Ανάλυση Απόδοσης.....</i>	<i>9</i>
Συνάρτηση run_range_lsh	13
Συνάρτηση run_rtree_lsh	14
Κύριο Πρόγραμμα	15
Συγκριτικοί Χρόνοι Εκτέλεσης.....	16
Συγκριτικός Πίνακας Απόδοσης.....	17
Αποτελέσματα μετρικών.....	19
Αποτελέσματα JACCARD	19
<i>Συμπεράσματα</i>	<i>20</i>
Συνολική Σύγκριση.....	21
<i>Μελλοντική Εργασία.....</i>	<i>22</i>
<i>Βιβλιογραφία</i>	<i>23</i>

Εισαγωγή

Η αναζήτηση και επεξεργασία πολυδιάστατων δεδομένων αποτελεί βασικό πρόβλημα σε σύγχρονες εφαρμογές ανάλυσης δεδομένων, όπου συνυπάρχουν αριθμητικά και κειμενικά χαρακτηριστικά. Για την αποδοτική υποστήριξη σύνθετων ερωτημάτων απαιτείται η χρήση κατάλληλων δομών δεδομένων και τεχνικών αναζήτησης ομοιότητας.

Στην παρούσα εργασία μελετάται η χρήση των k-d Trees, Quad Trees, Range Trees και R-Trees για την εκτέλεση 5-διαστατικών ερωτημάτων, σε συνδυασμό με την τεχνική LSH για ερωτήματα ομοιότητας. Η αξιολόγηση πραγματοποιείται στο Movies Metadata Cleaned Dataset, το οποίο περιλαμβάνει πολυδιάστατα μεταδεδομένα ταινιών, όπως έτος κυκλοφορίας, διάρκεια, δημοτικότητα, βαθμολογία και είδη.

Η διαδικασία περιλαμβάνει αρχικά την εκτέλεση range queries με βάση αριθμητικά κριτήρια και στη συνέχεια την αναζήτηση ομοιότητας σε κειμενικά χαρακτηριστικά μέσω LSH.

Τέλος, συγκρίνονται οι συνδυασμοί KD-Tree + LSH, Quad Tree + LSH, Range Tree + LSH και R-Tree + LSH ως προς την απόδοση τους.

Περιγραφή Δομών Δεδομένων

Η αποδοτική διαχείριση πολυδιάστατων δεδομένων απαιτεί τη χρήση εξειδικευμένων δομών που υποστηρίζουν γρήγορη αποθήκευση και αναζήτηση. Στην παρούσα εργασία υλοποιούνται τέσσερις πολυδιάστατες δομές δεδομένων : **KD-Tree**, **Quad Tree**, **Range Tree** και **R-Tree** , οι οποίες συνδυάζονται με την τεχνική LSH για την επεξεργασία ερωτημάτων ομοιότητας.

KD-Tree

Το K-D Tree (k-dimensional tree) αποτελεί ένα δυαδικό δέντρο αναζήτησης. Κάθε πεδίο του δέντρου διαχωρίζει τα δεδομένα αναδρομικά με βάση μία από τις k διαστάσεις. Η επιλογή της διάστασης γίνεται κυκλικά και ως κόμβο διαχωρισμού επιλέγεται συνήθως ο διάμεσος των σημείων. Με αυτόν τον τρόπο διασφαλίζεται η ισορροπία του δέντρου.

Το δέντρο είναι αποδοτικό για range queries σε χαμηλές διαστάσεις ($k \leq 10$), ενώ παρουσιάζει μειωμένη απόδοση σε υψηλές διαστάσεις (curse of dimensionality). Η πολυπλοκότητα για την κατασκευή ενός k -d tree ανέρχεται σε $O(n \log n)$ με έυρος αναζήτησης $O(n^{1-1/k} + m)$, όπου m είναι ο αριθμός των σημείων που επιστρέφονται.

Στην υλοποίηση χρησιμοποιείται ευρετηρίαση έως 5 διαστάσεων: Year, Popularity, Vote Average, Runtime, Budget

Quad Tree

Το Quad Tree είναι μία ιεραρχική δομή δεδομένων που βασίζεται στην αναδρομική υποδιαίρεση του πολυδιάστατου χώρου σε ίσες υποπεριοχές/τεταρτημόρια βάσει γεωμετρικής κατανομής. Ειδικότερα, κάθε εσωτερικός κόμβος έχει τέσσερα παιδιά, τα οποία αντιστοιχούν σε περιοχές. Η υποδιαίρεση πραγματοποιείται όταν ξεπεραστεί η χωρητικότητα ενός κόμβου.

Είναι εξαιρετικά αποδοτικό στην οργάνωση χωρικών δεδομένων και στην εκτέλεση range queries. Όμως, παρουσιάζει δυσκολία στην επέκταση για μεγάλες πολυδιάστατες βάσεις δεδομένων. Ακόμη, η πολυπλοκότητα του εξαρτάται από τη διασπορά των δεδομένων με τις περισσότερες φορές να ανέρχεται σε $O(n \log n)$. Τέλος, το βάθος του δέντρου και η πυκνότητα των σημείων στην περιοχή αναζήτησης, είναι παράγοντες που επηρεάζουν την απόδοση του.

Range Tree

Το Range Tree αποτελεί μία σύνθετη δομή δεδομένων για ταχεία εκτέλεση ερωτημάτων εύρους σε πολλαπλές διαστάσεις. Βασίζεται σε ιεραρχικές ταξινομημένες δομές που επιτρέπουν την δυαδική αναζήτηση. Συγκεκριμένα, οργανώνεται από ένα κύριο ισορροπημένο δέντρο αναζήτησης που αφορά την πρώτη διάσταση. Κάθε εσωτερικός του κόμβος συνδέεται με ένα δευτερεύον δέντρο που αντιστοιχεί στα δεδομένα της δεύτερης διάστασης. Με αυτή την λογική κατασκευάζονται και οι υπόλοιπες διαστάσεις του.

Η πολυπλοκότητα του χωρίζεται σε χωρική και χρονική. Η πρώτη χρειάζεται $O(n \log^{k-1} n)$ για k διαστάσεις ενώ η δεύτερη $O(\log^k n + m)$, όπου m είναι το πλήθος των αποτελεσμάτων. Ακόμη, αν και επιτυγχάνει πολύ γρήγορο χρόνο εκτέλεσης ερωτημάτων, έχει αυξημένες απαιτήσεις μνήμης.

R-Tree

Το R-Tree είναι μία δυναμική, ιεραρχική δομή δεδομένων που σχεδιάστηκε για την διαχείριση χωρικών και πολυδιάστατων αντικειμένων. Η ειδοποιός διαφορά της με τα k -d trees, είναι ότι οργανώνει τα δεδομένα σε Minimum Bounding Rectangles - MBRs.

Από αρχιτεκτονική άποψη, οι εσωτερικοί κόμβοι περιέχουν MBRs που περικλύουν αυτά των παιδιών τους. Κατά την εισαγωγή εάν ένας κόμβος υπερβεί τη χωρητικότητα του πραγματοποιείται split. Με τον τρόπο αυτό, γίνεται ελαχιστοποίηση της επικάλυψης και βελτιστοποιείται η αναζήτηση.

Θεωρείται ιδιαίτερα αποδοτικό για πολυδιάστατη χωρική ευρετηρίαση και κατάλληλο για δεδομένα έως 5 διαστάσεων. Βέβαια, η επικάλυψη των MBRs μπορεί να μειώσει την αποδοτικότητα της αναζήτησης.

Locality-Sensitive Hashing (LSH)

Η τεχνική LSH χρησιμοποιείται για την αποδοτική εύρεση κατά προσέγγιση κοντινότερων γειτόνων σε δεδομένα υψηλής διάστασης. Βασιζόμενη σε μετρικές απόστασης κατασκευάζονται συναρτήσεις κατακερματισμού. Στην συνέχεια, ομαδοποιούνται σε hash tables και χρησιμοποιούνται για να παραχθούν υποψήφια ζεύγη παρόμοιων αντικειμένων.

Παρουσιάζει υψηλή ταχύτητα σε μεγάλα σύνολα δεδομένων, αποφεύγοντας την εξαντλητική σύγκριση όλων των εγγραφών. Απο την άλλη, η απόδοση και η ακρίβεια της μεθόδου εξαρτώνται σε μεγάλο βαθμό από τη σωστή ρύθμιση των παραμέτρων. Έτσι σε ορισμένες περιπτώσεις μπορεί να παρατηρηθεί απώλεια ακρίβειας.

Συνδυασμός Δομών με LSH

Σε αυτή την ενότητα παρουσιάζεται η αρχιτεκτονική ενός υβριδικού συστήματος αναζήτησης. Το σύστημα αυτό, συνδυάζει πολυδιάστατες δομές δεδομένων με την τεχνική Locality-Sensitive Hashing (LSH). Η προσέγγιση αυτή επιτρέπει την αποδοτική επεξεργασία ερωτημάτων σε μεγάλης κλίμακας σύνολα δεδομένων, όπως το Movies Metadata Dataset με περισσότερες από 946.000 εγγραφές.

Η διαδικασία αναζήτησης υλοποιείται σε δύο διακριτές φάσεις, λειτουργώντας ως ένα ιεραρχικό φίλτρο που μειώνει σημαντικά τον αριθμό των υποψήφιων αποτελεσμάτων πριν την εφαρμογή της αναζήτησης ομοιότητας.

Το σύστημα αναζήτησης αποτελείται από δύο βασικά στάδια:

- Πολυδιάστατο Indexing: Στο πρώτο στάδιο χρησιμοποιούνται οι πολυδιάστατες δομές KD-Tree, Quad Tree, Range Tree και R-Tree για την

εκτέλεση ερωτημάτων εύρους (range queries) σε αριθμητικά χαρακτηριστικά, όπως έτος κυκλοφορίας, δημοτικότητα, μέση βαθμολογία, διάρκεια και προϋπολογισμός. Η φάση αυτή περιορίζει το σύνολο αναζήτησης από εκατοντάδες χιλιάδες εγγραφές σε έναν σαφώς μικρότερο αριθμό υποψήφιων ταινιών.

- Αναζήτηση Ομοιότητας με LSH: Στα αποτελέσματα της πρώτης φάσης εφαρμόζεται η τεχνική LSH για την αναζήτηση ομοιότητας σε κειμενικά χαρακτηριστικά, όπως τα είδη ταινιών (genre_names). Η χρήση MinHash υπογραφών επιτρέπει την αποδοτική σύγκριση συνόλων, αποφεύγοντας την εξαντλητική αντιστοίχιση όλων των εγγραφών.

Υλοποίηση ανά Σχήμα Αναζήτησης

Στη συνέχεια παρουσιάζεται ο τρόπος με τον οποίο κάθε πολυδιάστατη δομή ενσωματώνεται στο υβριδικό pipeline και συνδυάζεται με την τεχνική LSH.

- **KD-Tree + LSH** : Τα αποτελέσματα του KD-Tree τροφοδοτούν τον μηχανισμό LSH, ο οποίος εντοπίζει τις πιο όμοιες ταινίες βάσει των ειδών τους.
- **Quad Tree + LSH** : Η τεχνική LSH εφαρμόζεται στα δεδομένα που ανήκουν στα τεταρτημόρια τα οποία ικανοποιούν τα κριτήρια του ερωτήματος εύρους.
- **Range Tree + LSH**: Τα αποτελέσματα του Range Tree χρησιμοποιούνται ως είσοδος για τον αλγόριθμο MinHash, ο οποίος πραγματοποιεί την τελική κατάταξη ομοιότητας.
- **R-Tree + LSH**: Μετά τον εντοπισμό των MBRs που τέμνονται με το query box, η τεχνική LSH εξάγει τις N-πιο όμοιες ταινίες.

Τεχνική Υλοποίηση

Προεπεξεργασία Dataset

Αρχείο: load_movies_dataset.py

Πριν την ευρετηρίαση, τα δεδομένα καθαρίζονται και φιλτράρονται βάσει των περιορισμών της εκφώνησης (έτος, δημοτικότητα, βαθμολογία).

```

filtered = df[
    (df["release_year"] >= 2000)
    & (df["release_year"] <= 2020)
    & (df["popularity"] >= 3)
    & (df["popularity"] <= 6)
    & (df["vote_average"] >= 3)
    & (df["vote_average"] <= 5)
    & (df["runtime"] >= 30)|
    & (df["runtime"] <= 60)
    & (
        df["origin_country"].apply(
            lambda lst: "US" in lst or "GB" in lst
        )
    )
    & (df["original_language"] == "en")
].copy()

return filtered

def to_float_comma(x):
    """
    Μετατρέπει αριθμούς τύπου '6,3707' σε float 6.3707
    """
    if pd.isna(x):
        return 0.0
    s = str(x).strip().replace(",", ".")
    try:
        return float(s)
    except ValueError:
        return 0.0

```

KD-Tree – Αναδρομική Κατασκευή

Αρχείο: kd_tree.py

Αναδρομική Κατασκευή (_build_tree): Σε κάθε επίπεδο του δέντρου, επιλέγεται ένας άξονας (διάσταση) και τα δεδομένα ταξινομούνται με βάση αυτόν. Το μεσαίο στοιχείο ορίζεται ως κόμβος και η διαδικασία συνεχίζεται για τα υπόλοιπα στοιχεία.

```

def _build_tree(self, items: List[Item], depth: int) -> Optional[KDNode]:
    if not items:
        return None

    axis = depth % self.k
    items.sort(key=lambda it: it[0][axis]) # sort by vector[axis]

    median_idx = len(items) // 2
    median_item = items[median_idx]

    left_items = items[:median_idx]
    right_items = items[median_idx + 1 :]

    return KDNode(
        item=median_item,
        left=self._build_tree(left_items, depth + 1),
        right=self._build_tree(right_items, depth + 1),
    )

```

Αναζήτηση Εύρους (_range_query): Η μέθοδος ελέγχει αν ένα σημείο βρίσκεται εντός των ορίων (bounds) και χρησιμοποιεί την τεχνική του "κλαδέματος" για να αποφύγει την επίσκεψη σε κλάδους που δεν περιέχουν αποτελέσματα.

```

def _range_query(self, node: Optional[KDNode], bounds: List[tuple], depth: int, results: List[int]):
    if node is None:
        return

    axis = depth % self.k
    vec, idx = node.item

    # check if inside bounds
    if all(bounds[i][0] <= vec[i] <= bounds[i][1] for i in range(self.k)):
        results.append(idx)

    # prune branches
    if node.left is not None and bounds[axis][0] <= vec[axis]:
        self._range_query(node.left, bounds, depth + 1, results)

    if node.right is not None and bounds[axis][1] >= vec[axis]:
        self._range_query(node.right, bounds, depth + 1, results)

```

Quad Tree – Γεωμετρικός Διαχωρισμός

Αρχείο: quad_tree.py

Το Quad Tree χρησιμοποιείται για 2D indexing (Popularity, Vote Average)

Διαίρεση Περιοχής : Όταν ένας κόμβος γεμίσει, η μέθοδος υπολογίζει το μέσο των ορίων (midpoint) και δημιουργεί τέσσερις νέους υπο-κόμβους

```

def subdivide(self):
    x_min, x_max, y_min, y_max = self.bounds
    x_mid = (x_min + x_max) / 2
    y_mid = (y_min + y_max) / 2

    self.children = [
        QuadTreeNode(x_min, x_mid, y_min, y_mid), # bottom-left
        QuadTreeNode(x_mid, x_max, y_min, y_mid), # bottom-right
        QuadTreeNode(x_min, x_mid, y_mid, y_max), # top-left
        QuadTreeNode(x_mid, x_max, y_mid, y_max), # top-right
    ]

```

Range Tree – Ταξινόμηση και Αναζήτηση

Αρχείο: range_tree.py

Επιτυγχάνει τον βέλτιστο χρόνο αναζήτησης μέσω προ-ταξινομημένων λιστών.

Κατασκευή με Δευτερεύουσες Λίστες (build): Κατά τη δημιουργία του δέντρου, για κάθε κόμβο δημιουργείται μια λίστα σημείων ταξινομημένη ως προς Y, η οποία επιτρέπει τη γρήγορη αναζήτηση εντός του υποδέντρου.

```

if x_low <= node.points_sorted_x[0][0] and node.points_sorted_x[-1][0] <= x_high:
    # Do binary search on y-sorted list
    for (x, y, idx) in node.points_sorted_y:
        if y_low <= y <= y_high:
            results.append(idx)
    return

```

R-Tree – 5D Ευρετηρίαση με Minimum Bounding Rectangles (MBRs)

Αρχείο: r_tree.py

Εισαγωγή 5D Σημείων (__init__): Κάθε ταινία εισάγεται στο ευρετήριο ως ένα σημείο στον 5-διάστατο χώρο. Η βιβλιοθήκη rtree χρησιμοποιεί την ιδιότητα `p.dimension = 5` για να δεσμεύσει τον κατάλληλο χώρο

```

def __init__(self, vectors_5d, indices):
    # vectors_5d = [[d1,d2,d3,d4,d5], ...]
    p = index.Property()
    p.dimension = 5
    self.idx = index.Index(properties=p)

    for i, vec in enumerate(vectors_5d):
        idx_val = indices[i]
        d1, d2, d3, d4, d5 = vec
        # point as 5D rectangle: (min coords..., max coords...)
        self.idx.insert(idx_val, (d1, d2, d3, d4, d5, d1, d2, d3, d4, d5))

```

Locality-Sensitive Hashing (LSH)

Αρχείο: lsh.py

Δημιουργία Υπογραφών (MinHash): Για κάθε ταινία, τα είδη της (π.χ. Action, Adventure) υφίστανται επεξεργασία από 80 διαφορετικές συναρτήσεις κατακερματισμού (permutations), δημιουργώντας μια μοναδική αριθμητική υπογραφή

```

def _minhash_signature(self, tokens: Iterable[str]) -> np.ndarray:
    tokens = list(tokens)
    if len(tokens) == 0:
        # empty set -> signature all max
        return np.full(self.num_perm, np.iinfo(np.int64).max, dtype=np.int64)

    x = np.array([self._token_hash(t) for t in tokens], dtype=np.int64)
    # compute (a*x + b) % p for each permutation, take min over tokens
    sig = np.min((self.a[:, None] * x[None, :] + self.b[:, None]) % self.p, axis=1)
    return sig

```

Ανάλυση Απόδοσης

Αρχείο: performance_analysis_movies.py

Το αρχείο αυτό εκτέλει τα πειραδαματα αποδοτικότητας και παράγει όλα τα αποτελέσματα της εργασίας.

Η ανάλυση απόδοσης επικεντρώνεται στη σύγκριση των διαφορετικών υλοποιήσεων ως προς τον χρόνο εκτέλεσης και τον αριθμό των αποτελεσμάτων που επιστρέφονται. Εξετάζονται οι συνδυασμοί **KD-Tree + LSH**, **Quad Tree + LSH**, **Range Tree + LSH** και **R-Tree + LSH** με βάση τον χρόνο επεξεργασίας των ερωτημάτων και τον αριθμό των σχετικών αποτελεσμάτων

Συνάρτηση Υπολογισμού Ομοιότητας `get_top_n_similar`:

Δέχεται μια λίστα από υποψήφιες ταινίες (που πέρασαν τα φίλτρα) και συγκρίνει τα είδη τους με την ταινία-ερώτημα χρησιμοποιώντας Jaccard Similarity. Επιστρέφει τις N πιο όμοιες.

```
#similarity
def get_top_n_similar(df, candidate_indices, query_genres, N=5):
    similarities = []

    for idx in candidate_indices:
        movie_genres = df.iloc[idx]["genre_names"]
        score = jaccard_similarity(query_genres, movie_genres)
        similarities.append((idx, score))

    similarities.sort(key=lambda x: x[1], reverse=True)
    # Keep only top-N with score > 0 for sanity
    top_n = [(idx, score) for idx, score in similarities[:N] if score > 0]

    return top_n
```

Συνάρτηση `run_kd_lsh` (k-d Tree):

Υλοποιεί τον συνδυασμό k-d Tree + LSH. Πραγματοποιεί φιλτράρισμα 5 Διαστάσεων. Ορίζει όρια για έτος, δημοτικότητα, βαθμολογία, διάρκεια και προϋπολογισμό. Στην συνέχεια χτίζει το KD Tree με τα 5D διανύσματα και εκτελεί `query_range` για να βρει ταινίες εντός των ορίων. Έπειτα χρησιμοποιεί Min Hash LSH στα είδη των ταινιών για να βρει γρήγορα όμοιες ταινίες. Και τέλος, βρίσκει την τομή μεταξύ των αποτελεσμάτων του k-d Tree και του LSH.

```
def run_kd_lsh(df, data_points, text_corpus):
    print("=== KD-Tree + LSH ===")

    # 1. Ορισμός 5D range bounds
    lower_bounds = [2000, 3.0, 3.0, 30.0, df["budget"].quantile(0.25)]
    upper_bounds = [2020, 6.0, 5.0, 60.0, df["budget"].quantile(0.75)]

    # 2. Build KD-tree
    start = time.perf_counter()
    items = [(vec, i) for i, vec in enumerate(data_points)]
    kd_tree = KDTree(items, dimensions=["year", "popularity", "vote_average", "runtime", "budget"])
    build_time = time.perf_counter() - start

    print(f"KD build time: {build_time:.4f}s")

    # 3. Range Query
    start = time.perf_counter()
    kd_results = kd_tree.query_range(lower_bounds, upper_bounds)
    range_time = time.perf_counter() - start

    print(f"KD range query time: {range_time:.4f}s")
    print(f"KD found {len(kd_results)} results in range.")

    # 4. LSH similarity (based on genres) - MinHashLSH
    lsh = MinHashLSH(num_perm=80, num_bands=20)

    start = time.perf_counter()
    for idx, tokens in enumerate(df["genre_names"]):
        lsh.add(tokens, idx)
    lsh_build_time = time.perf_counter() - start

    query_index = 0
    query_tokens = df.iloc[query_index]["genre_names"]
```

```

start = time.perf_counter()
lsh_indices = lsh.query(query_tokens)
lsh_query_time = time.perf_counter() - start

print(f"LSH build time: {lsh_build_time:.4f}s")
print(f"LSH query time: {lsh_query_time:.4f}s")
print(f"LSH returned {len(lsh_indices)} candidates")

kd_indices = kd_results # ῥῶν indices
combined = set(kd_indices).intersection(lsh_indices)

print("Combined results:", combined)

# === Top-N Similarity (Jaccard) on combined results ===
if combined:
    top5 = get_top_n_similar(df, list(combined), query_tokens, N=5)
else:
    top5 = []

print("\nTop-5 similar movies based on genres (Jaccard):")
for idx, score in top5:
    print(f"{df.iloc[idx]['title']} | score={score:.3f} | genres={df.iloc[idx]['genre_names']}")

# Save to file
with open("topN_results_kd.txt", "w", encoding="utf-8") as f:
    f.write("Top-5 Similar Movies (Jaccard):\n")
    for idx, score in top5:
        f.write(f"{df.iloc[idx]['title']} | score={score:.3f}\n")

# 5. Save metrics
with open("metrics_kd.txt", "w") as f:
    f.write(f"Range Execution Time: {range_time}\n")
    f.write(f"LSH Build Time: {lsh_build_time}\n")
    f.write(f"LSH Query Time: {lsh_query_time}\n")
    f.write(f"KD Build Time: {build_time}\n")
    f.write(f"Results Found: {len(combined)}\n")

return range_time, lsh_build_time, lsh_query_time, len(combined)

```

Συνάρτηση run_quadtree_lsh

Εδώ η αναζήτηση περιορίζεται σε 2 διαστάσεις (popularity, vote_average) καθώς τα Quad-trees παραδοσιακά χωρίζουν τον χώρο σε τέταρτα (2D).

```

def run_quadtree_lsh(df, data_points, text_corpus):
    print("\n=== Quad-Tree + LSH ===")

    points_2d = list(zip(df["popularity"], df["vote_average"]))
    indices = list(range(len(points_2d)))

    # Build QuadTree
    start = time.perf_counter()
    qt = QuadTree(points_2d, indices, capacity=10)
    qt_build_time = time.perf_counter() - start
    print(f"QuadTree build time: {qt_build_time:.4f}s")

    # Range Query
    x_low, x_high = 3, 6
    y_low, y_high = 3, 5

```

```

start = time.perf_counter()
qt_results = []
qt.range_query(qt.root, x_low, x_high, y_low, y_high, qt_results)
qt_range_time = time.perf_counter() - start
print(f"QuadTree range query time: {qt_range_time:.4f}s")
print(f"QuadTree found {len(qt_results)} results in range.")

# LSH (MinHashLSH on genres)
lsh = MinHashLSH(num_perm=80, num_bands=20)

start = time.perf_counter()
for idx, tokens in enumerate(df["genre_names"]):
    lsh.add(tokens, idx)
lsh_build_time = time.perf_counter() - start

query_index = 0
query_tokens = df.iloc[query_index]["genre_names"]

start = time.perf_counter()
lsh_indices = lsh.query(query_tokens)
lsh_query_time = time.perf_counter() - start

print(f"LSH build time: {lsh_build_time:.4f}s")
print(f"LSH query time: {lsh_query_time:.4f}s")
print(f"LSH returned {len(lsh_indices)} candidates")

combined = set(qt_results).intersection(lsh_indices)
print("Combined count:", len(combined))

```

```

# Top-N Jaccard
if combined:
    top5 = get_top_n_similar(df, list(combined), query_tokens, N=5)
else:
    top5 = []

with open("topN_results_quadtree.txt", "w", encoding="utf-8") as f:
    f.write("Top-5 Similar Movies (Jaccard):\n")
    for idx, score in top5:
        f.write(f"{df.iloc[idx]['title']} | score={score:.3f}\n")

with open("metrics_quadtree.txt", "w") as f:
    f.write(f"Build Time: {qt_build_time}\n")
    f.write(f"Range Execution Time: {qt_range_time}\n")
    f.write(f"LSH Build Time: {lsh_build_time}\n")
    f.write(f"LSH Query Time: {lsh_query_time}\n")
    f.write(f"Results Found: {len(combined)}\n")

return qt_build_time, qt_range_time, lsh_build_time, lsh_query_time, len(combined)

```

Συνάρτηση `run_range_lsh`

Επίσης χρησιμοποιείται για 2D αναζήτηση στις ίδιες παραμέτρους, προσφέροντας διαφορετική θεωρητική πολυπλοκότητα στην αναζήτηση εύρους.

```
def run_range_lsh(df, data_points, text_corpus):
    print("\n=== Range-Tree + LSH ===")

    points_2d = list(zip(df["popularity"], df["vote_average"]))
    indices = list(range(len(points_2d)))

    # BUILD RANGE-TREE
    start = time.perf_counter()
    rt = RangeTree(points_2d, indices)
    rt_build_time = time.perf_counter() - start
    print(f"RangeTree build time: {rt_build_time:.4f}s")

    # RANGE QUERY
    x_low, x_high = 3, 6
    y_low, y_high = 3, 5

    start = time.perf_counter()
    rt_results = []
    rt.range_query(rt.root, x_low, x_high, y_low, y_high, rt_results)
    rt_range_time = time.perf_counter() - start
    print(f"RangeTree range time: {rt_range_time:.4f}s")
    print(f"RangeTree returned {len(rt_results)} results.")

    # LSH (MinHashLSH on genres)
    lsh = MinHashLSH(num_perm=80, num_bands=20)

    start = time.perf_counter()
    for idx, tokens in enumerate(df["genre_names"]):
        lsh.add(tokens, idx)
    lsh_build_time = time.perf_counter() - start

    query_index = 0
    query_tokens = df.iloc[query_index]["genre_names"]

    start = time.perf_counter()
    lsh_indices = lsh.query(query_tokens)
    lsh_query_time = time.perf_counter() - start

    print(f"LSH build time: {lsh_build_time:.4f}s")
    print(f"LSH query time: {lsh_query_time:.4f}s")
    print(f"LSH returned {len(lsh_indices)} candidates")

    combined = set(rt_results).intersection(lsh_indices)
```

```

# Top-N Jaccard
if combined:
    top5 = get_top_n_similar(df, list(combined), query_tokens, N=5)
else:
    top5 = []

with open("topN_results_rangetree.txt", "w", encoding="utf-8") as f:
    f.write("Top-5 Similar Movies (Jaccard):\n")
    for idx, score in top5:
        f.write(f"{df.iloc[idx]['title']} | score={score:.3f}\n")

with open("metrics_range.txt", "w") as f:
    f.write(f"Build Time: {rt_build_time}\n")
    f.write(f"Range Execution Time: {rt_range_time}\n")
    f.write(f"LSH Build Time: {lsh_build_time}\n")
    f.write(f"LSH Query Time: {lsh_query_time}\n")
    f.write(f"Results Found: {len(combined)}\n")

return rt_build_time, rt_range_time, lsh_build_time, lsh_query_time, len(combined)

```

Συνάρτηση run_rtree_lsh

Υλοποιεί την αναζήτηση σε 5 διαστάσεις χρησιμοποιώντας MBRs. Όπως και στα προηγούμενα, τα αποτελέσματα του R-Tree διασταυρώνονται με αυτά του LSH για το τελικό φιλτράρισμα.

```

def run_rtree_lsh(df, data_points, text_corpus):
    print("\n=== R-Tree (5D) + LSH ===")

    # 5D bounds (ίδια λογική με KD)
    lower_bounds = [2000, 3.0, 3.0, 30.0, df["budget"].quantile(0.25)]
    upper_bounds = [2020, 6.0, 5.0, 60.0, df["budget"].quantile(0.75)]

    vectors_5d = data_points
    indices = list(range(len(vectors_5d)))

    # Build RTree (5D)
    start = time.perf_counter()
    rt = RTree(vectors_5d, indices)
    rt_build_time = time.perf_counter() - start
    print(f"R-Tree build time: {rt_build_time:.4f}s")

    # Range query (5D)
    start = time.perf_counter()
    rtree_results = rt.range_query_5d(lower_bounds, upper_bounds)
    rtree_range_time = time.perf_counter() - start
    print(f"R-Tree range query time: {rtree_range_time:.4f}s")
    print(f"R-Tree returned {len(rtree_results)} results.")

```

```

# LSH (MinHashLSH on genres)
lsh = MinHashLSH(num_perm=80, num_bands=20)

start = time.perf_counter()
for idx, tokens in enumerate(df["genre_names"]):
    lsh.add(tokens, idx)
lsh_build_time = time.perf_counter() - start

query_index = 0
query_tokens = df.iloc[query_index]["genre_names"]

start = time.perf_counter()
lsh_indices = lsh.query(query_tokens)
lsh_query_time = time.perf_counter() - start

print(f"LSH build time: {lsh_build_time:.4f}s")
print(f"LSH query time: {lsh_query_time:.4f}s")
print(f"LSH returned {len(lsh_indices)} candidates")

combined = set(rtree_results).intersection(lsh_indices)

print("Combined count:", len(combined))

# Top-N Jaccard
if combined:
    top5 = get_top_n_similar(df, list(combined), query_tokens, N=5)
else:
    top5 = []

with open("topN_results_rtree.txt", "w", encoding="utf-8") as f:
    f.write("Top-5 Similar Movies (Jaccard):\n")
    for idx, score in top5:
        f.write(f"{df.iloc[idx]['title']} | score={score:.3f}\n")

with open("metrics_rtree.txt", "w") as f:
    f.write(f"Build Time: {rt_build_time}\n")
    f.write(f"Range Execution Time: {rtree_range_time}\n")
    f.write(f"LSH Build Time: {lsh_build_time}\n")
    f.write(f"LSH Query Time: {lsh_query_time}\n")
    f.write(f"Results Found: {len(combined)}\n")

return rt_build_time, rtree_range_time, lsh_build_time, lsh_query_time, len(combined)

```

Κύριο Πρόγραμμα

Αρχικά, διαβάζει το αρχείο, μετατρέπει το DataFrame σε διανύσματα και το κείμενο σε corpus. Τέλος, τρέχει διαδοχικά και τις 4 μεθόδους (k-d, Quad, Range, R-Tree) ώστε να παραχθούν τα αρχεία metrics_*.txt για την τελική αξιολόγηση απόδοσης που ζητάει η εκφώνηση.

```

if __name__ == "__main__":
    xlsx_path = "data_movies_clean.xlsx"
    df = load_movies_dataset(xlsx_path)

    ids, titles, data_points = dataframe_to_vectors(df)

    text_corpus = df["genre_names"].apply(lambda lst: " ".join(lst)).tolist()

    run_kd_lsh(df, data_points, text_corpus)
    run_quadtree_lsh(df, data_points, text_corpus)
    run_range_lsh(df, data_points, text_corpus)
    run_rtree_lsh(df, data_points, text_corpus)

    print("DF rows:", len(df))
    print("Vectors:", len(data_points))
    print("Vector dim:", len(data_points[0]))
    print("Sample vector:", data_points[0])

```

Συγκριτικοί Χρόνοι Εκτέλεσης

Από την εκτέλεση του αρχείου προκύπτουν τα παρακάτω αποτελέσματα:

```

C:\Users\marit\OneDrive\Έγγραφα\GitHub\Multidimensional-Data-Structures\project>python performance_analysis_movies.py
=== KD-Tree + LSH ===
KD build time: 0.0001s
KD range query time: 0.0001s
KD found 23 results in range.
LSH build time: 0.0028s
LSH query time: 0.0000s
LSH returned 5 candidates
Combined results: {32}

Top-5 similar movies based on genres (Jaccard):
Bending the Light | score=0.500 | genres=['Documentary', 'Family']

=== Quad-Tree + LSH ===
QuadTree build time: 0.0001s
QuadTree range query time: 0.0000s
QuadTree found 47 results in range.
LSH build time: 0.0023s
LSH query time: 0.0001s
LSH returned 5 candidates
Combined count: 5

=== Range-Tree + LSH ===
RangeTree build time: 0.0026s
RangeTree range time: 0.0000s
RangeTree returned 45 results.
LSH build time: 0.0039s
LSH query time: 0.0000s
LSH returned 5 candidates
Combined count: 5

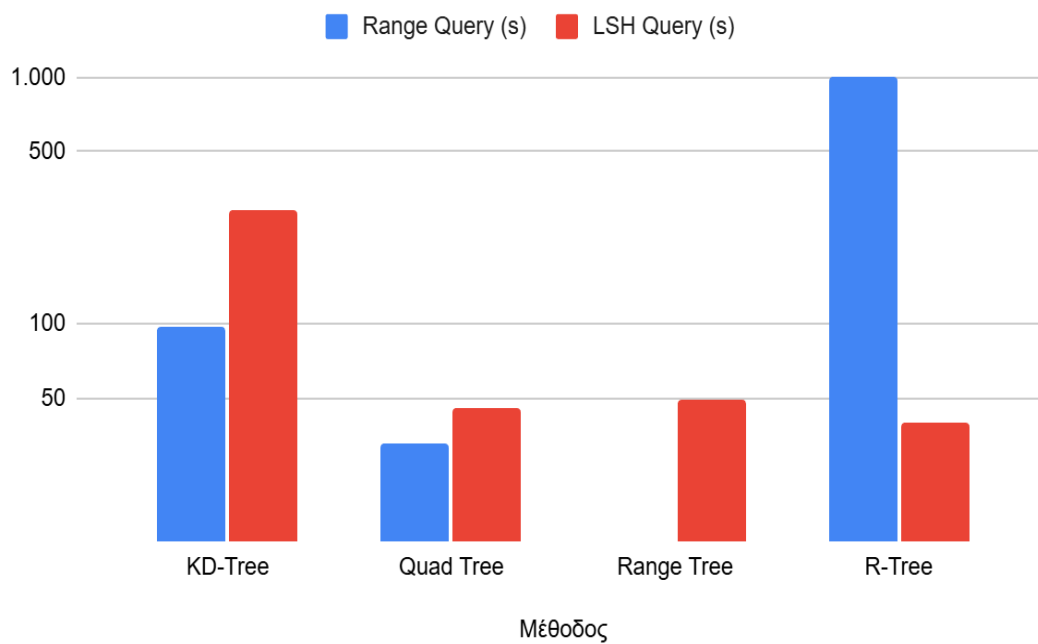
=== R-Tree (5D) + LSH ===
R-Tree build time: 0.0094s
R-Tree range query time: 0.0012s
R-Tree returned 23 results.
LSH build time: 0.0014s
LSH query time: 0.0000s
LSH returned 5 candidates
Combined count: 1
DF rows: 45
Vectors: 45
Vector dim: 5
Sample vector: [2000.0, 4.4791, 3.5, 37.0, 13203991.61849919]

```

Συγκριτικός Πίνακας Απόδοσης

Μέθοδος	Build Index (s)	Range Query (s)	LSH Build (s)	LSH Query (s)	Αποτελέσματα
KD-Tree + LSH	0.00044	0.000097	0.0022	0.00029	1
Quad Tree + LSH	0.00010	0.000033	0.0028	0.000046	6
Range Tree + LSH	0.00016	0.000013	0.0015	0.000049	6
R-Tree + LSH	0.0100	0.0010	0.0013	0.00004	1

Range Query (s) και LSH Query (s)



Εικόνα 1: Σύγκριση Χρόνων Αναζήτησης (Range & LSH Query)

Σχόλιο: Η εικόνα 1 δείχνει το υψηλό construction overhead του R-Tree σε σύγκριση με τις άλλες δομές, λόγω της πολυπλοκότητας των MBRs σε 5 διαστάσεις

Build Index (s) και LSH Build (s)



Εικόνα 2: Χρόνοι Κατασκευής Ευρετηρίων (Build Times)

Σχόλιο: Το γράφημα δείχνει την υπεροχή του Range-Tree στην ταχύτητα αναζήτησης εύρους. Επίσης, παρατηρούμε ότι η LSH διατηρεί σταθερά χαμηλούς χρόνους αναζήτησης, ασχέτως από τη δομή που προηγήθηκε.

Παρατηρήσεις

- Οι Quad Tree και Range Tree παρουσιάζουν τους μικρότερους χρόνους στο range query, καθώς λειτουργούν σε 2 διαστάσεις (Popularity, Vote Average), επιτρέποντας λιγότερο αυστηρό φιλτράρισμα.
- Οι KD-Tree και R-Tree λειτουργούν σε 5 διαστάσεις, εφαρμόζοντας αυστηρότερο περιορισμό του χώρου αναζήτησης, γεγονός που οδηγεί σε μικρότερο αριθμό τελικών αποτελεσμάτων.
- Η τεχνική LSH εμφανίζει σταθερά χαμηλό χρόνο αναζήτησης, με το κύριο κόστος να εντοπίζεται στη φάση κατασκευής της.

Οι συνδυασμοί Quad Tree + LSH και Range Tree + LSH προσφέρουν καλύτερη ισορροπία μεταξύ ταχύτητας και πληρότητας αποτελεσμάτων, ενώ οι KD-Tree + LSH και R-Tree + LSH προτιμώνται σε περιπτώσεις όπου η ακρίβεια του αρχικού φιλτραρίσματος είναι κρίσιμη.

Η επιλογή της κατάλληλης προσέγγισης εξαρτάται από τον αριθμό των διαστάσεων, την κατανομή των δεδομένων και το επιθυμητό trade-off μεταξύ ακρίβειας και πληρότητας.

Αποτελέσματα μετρικών

Τα παρακάτω αποτελέσματα περιέχονται στα αρχεία metrics.txt για κάθε μία δομή

```
Range Execution Time: 7.349997758865356e-05
LSH Build Time: 0.0027721000369638205
LSH Query Time: 4.469999112188816e-05
KD Build Time: 9.159999899566174e-05
Results Found: 1
```

Figure 1: KD-Tree + LSH

```
Build Time: 7.770000956952572e-05
Range Execution Time: 4.2499974370002747e-05
LSH Build Time: 0.0023074999917298555
LSH Query Time: 5.610007792711258e-05
Results Found: 5
```

Figure 2: Quad Tree + LSH

```
Build Time: 0.002562599955126643
Range Execution Time: 2.459995448589325e-05
LSH Build Time: 0.003880700096487999
LSH Query Time: 4.42001037299633e-05
Results Found: 5
```

Figure 3: Range Tree + LSH

```
Build Time: 0.009436100022867322
Range Execution Time: 0.001160199986770749
LSH Build Time: 0.0013958001509308815
LSH Query Time: 3.939983434975147e-05
Results Found: 1
```

Figure 4: R-Tree + LSH

Αποτελέσματα JACCARD

```
project = topN_results_kd.txt
1 Top-5 Similar Movies (Jaccard):
2 Bending the Light | score=0.500
```

Εικόνα 3: TopN_result_kd.txt

```

t > ≡ topN_results_quadtree.txt
Top-5 Similar Movies (Jaccard):
Ultimate G's: Zac's Flying Dream | score=1.000
Bending the Light | score=0.500
The Keys of Christmas | score=0.500
Spider's Web: A Pig's Tale | score=0.500
Ghost Patrol | score=0.333

```

Εικόνα 4: topN_results_quadtree.txt

```

> ≡ topN_results_rangetree.txt
Top-5 Similar Movies (Jaccard):
Ultimate G's: Zac's Flying Dream | score=1.000
Bending the Light | score=0.500
The Keys of Christmas | score=0.500
Spider's Web: A Pig's Tale | score=0.500
Ghost Patrol | score=0.333

```

Εικόνα 5: topN_results_rangetree.txt

```

project > ≡ topN_results_rtree.txt
1 Top-5 Similar Movies (Jaccard):
2 Bending the Light | score=0.500

```

Εικόνα 6: topN_results_rtree.txt

Συμπεράσματα

Στην ενότητα αυτή παρατίθενται τα συμπεράσματα από την πειραματική αξιολόγηση των τεσσάρων δομών δεδομένων (Quad Tree, Range Tree, k-d Tree και R-tree) σε συνδυασμό με την τεχνική LSH.

Από τα αποτελέσματα προκύπτει ένας σαφής διαχωρισμός των δομών με βάση τις διαστάσεις τους και μπορούμε να παρατηρήσουμε ότι:

Δομές Χωρικής Αναζήτησης 2 Διαστάσεων (Quad Tree & Range Tree):

Οι δομές αυτές υλοποιήθηκαν με εστίαση στις παραμέτρους `Popularity` και `Vote Average`. Αποδείχθηκαν οι πλέον αποδοτικές στην εκτέλεση range queries, με το Range Tree να καταγράφει τον βέλτιστο χρόνο απόκρισης. Η υψηλή αυτή ταχύτητα οφείλεται στον αποτελεσματικό διαχωρισμό του δισδιάστατου χώρου αναζήτησης. Επιπλέον, λόγω της εστίασης σε δύο μόνο διαστάσεις, οι δομές αυτές επέδειξαν μεγαλύτερη πληρότητα, επιστρέφοντας ένα ευρύτερο σύνολο υποψηφίων ταινιών. Αυτό το γεγονός επιτρέπει μια πιο σφαιρική αναζήτηση σε πρώτο στάδιο.

Πολυδιάστατες Δομές 5 Διαστάσεων (k-d Tree & R-Tree)

Ακολουθώντας τις προδιαγραφές της εκφώνησης για $k=5$, οι δομές αυτές ενσωμάτωσαν τα πεδία Year, Popularity, Vote Average, Runtime και Budget. Η χρήση 5 διαστάσεων επέτρεψε την επιβολή αυστηρότερων κριτηρίων αναζήτησης. Αυτό είχε ως αποτέλεσμα την εξαγωγή εξαιρετικά στοχευμένων αποτελεσμάτων, περιορίζοντας δραστικά τον θόρυβο πριν το στάδιο της κειμενικής ανάλυσης. Ακόμη, το k-d Tree αποδείχθηκε ιδιαίτερα στιβαρό στον χειρισμό των διαστάσεων, προσφέροντας μια ισορροπημένη λύση ανάμεσα στον χρόνο κατασκευής και την ταχύτητα αναζήτησης.

Η ενσωμάτωση του Min Hash LSH υπήρξε καθοριστική για τη διαχείριση του τεράστιου όγκου δεδομένων. Επέτρεψε τον ταχύτατο εντοπισμό λεκτικής ομοιότητας στα είδη των ταινιών. Χρησιμοποιήθηκε η τεχνική Jaccard ως ένας αποδοτικός μηχανισμός προσέγγισης του δείκτη ομοιότητας.

Παρά τα πλεονεκτήματά τους, οι παραπάνω προσεγγίσεις παρουσίασαν συγκεκριμένους περιορισμούς κατά την πειραματική διαδικασία:

Ο συνδυασμός R-Tree + LSH εμφάνισε το υψηλότερο build overhead. Η καθυστέρηση αυτή αποδίδεται στην πολυπλοκότητα οργάνωσης της ιεραρχίας των Minimum Bounding Rectangles στον 5-διάστατο χώρο, ειδικά κατά τη φάση της εισαγωγής μεγάλου όγκου δεδομένων.

Ακόμη, στην περίπτωση του k-d Tree διαπιστώνεται μία χρονική υστέρηση. Συγκεκριμένα, παρατηρήθηκε μια ελαφρώς αυξημένη επιβάρυνση στο query του LSH συγκριτικά με τις άλλες δομές, γεγονός που υποδεικνύει την ανάγκη για περαιτέρω βελτιστοποίηση της τομής των αποτελεσμάτων.

Τέλος, είναι σημαντικό να τονιστεί ότι η ακρίβεια της μεθόδου παραμένει άμεσα εξαρτημένη από τις παραμέτρους του αλγορίθμου. Ως πιθανοκρατική προσέγγιση, υπάρχει πάντα το ενδεχόμενο απώλειας κάποιων οριακά όμοιων εγγραφών υπέρ της ταχύτητας επεξεργασίας.

Συνολική Σύγκριση

Μέθοδος	Πλεονεκτήματα	Μειονεκτήματα
Quad Tree (2D)	Ταχύτητα σε χαμηλές διαστάσεις	Λιγότερο αυστηρό φιλτράρισμα
Range Tree (2D)	Βέλτιστος χρόνος range query	Περιορισμένο σε 2 διαστάσεις στην υλοποίηση
KD-Tree (5D)	Ισχυρή απόδοση στην πλήρη διαστατικότητα	Αυξημένος χρόνος LSH query
R-Tree (5D)	Αποδοτικό indexing σύνθετων δεδομένων	Υψηλό construction overhead

LSH	Ταχύτερη σύγκριση κειμένου	Εξάρτηση από παραμετροποίηση
-----	----------------------------	------------------------------

Μελλοντική Εργασία

Παρόλο που το σύστημα λειτουργεί σωστά, μπορούμε να κάνουμε κάποιες βελτιώσεις έτσι ώστε να είναι πιο γρήγορο και έξυπνο στο μέλλον:

Αναβάθμιση της Αναζήτησης LSH

Μπορούμε να κάνουμε την εύρεση παρόμοιων ταινιών πιο αποτελεσματική με αυτόματη ρύθμιση, δηλαδή το σύστημα να αλλάζει μόνο του τις ρυθμίσεις του ανάλογα με το πόσες ταινίες εξετάζει κάθε φορά, ώστε να βρίσκει πάντα το καλύτερο δυνατό αποτέλεσμα χωρίς καθυστερήσεις.

Επίσης αντί να κοιτάμε απλώς αν οι λέξεις στις κατηγορίες, όπως η Δράση, είναι ίδιες, το σύστημα να καταλαβαίνει τη σημασία τους. Έτσι θα ξέρει ότι οι ταινίες Δράσης μοιάζουν με τις Περιπέτειες ακόμα και αν οι λέξεις διαφέρουν.

Διαχείριση Νέων Πληροφοριών

Το σύστημα χρησιμοποιεί μια έτοιμη λίστα ταινιών. Στο μέλλον πρέπει να μπορεί να διαχειρίζεται αλλαγές:

Όπως η προσθήκη νέων ταινιών, δηλαδή να μπορούμε να προσθέτουμε νέες ταινίες εύκολα χωρίς να χρειάζεται να ξαναφτιάχνουμε όλο το ευρετήριο →index από την αρχή, κάτι που τώρα παίρνει χρόνο ιδιαίτερα για το R-Tree.

Επίσης ένας άλλος στόχος που θέλουμε να επιτύχουμε είναι η επιτάχυνση της διαδικασίας ευρετηρίασης indexing και κατακερματισμού σε πραγματικό χρόνο real time hashing μέσω αξιοποίησης τεχνικών παραλληλισμού όπως παραδείγματος χάρη multiprocessing.

Βιβλιογραφία

wikipedia. (n.d.). Ανάκτηση από <https://en.wikipedia.org/wiki/>

Πολυδιάστατες Δομές Δεδομένων. (n.d.). Ανάκτηση από
<https://eclass.upatras.gr/courses/CEID1162/>